DSP-BASED ACOUSTIC PATTERN ANALYSIS AND CLASSIFICATION

PROJECT REPORT

Platform: Texas Instruments TMS320C6745 DSP Kit

Language: Embedded C **IDE:** Code Composer Studio

Synthetic acoustic signal generation, time-domain feature extraction, and threshold-based classification

Student Name	GOPI GANESH J
Department	Electronics and Communication Engineering
Institution	PUDUCHERRY TECHNOLOGICAL UNIVERSITY
Academic Year	2026–27

Abstract

This project implements a DSP-based acoustic pattern analysis prototype in C for a Texas Instruments TMS320C6745 DSP development kit using Code Composer Studio (CCS). The program creates three synthetic audio-like signals in memory: a short decaying clap-like waveform, a steady sinusoidal whistle-like waveform, and low-amplitude pseudo-random background noise. For each signal, it computes RMS amplitude, peak amplitude, crest factor, and zero-crossing rate. A rule-based classifier assigns one of three labels—CLAP-LIKE, WHISTLE-LIKE, or BACKGROUND-LIKE—using fixed thresholds. The values and labels are printed to the CCS console. The implementation demonstrates a reproducible DSP processing chain; it does not capture live microphone audio or establish accuracy on real-world recordings.

Keywords: Digital Signal Processing, TMS320C6745, Embedded C, RMS amplitude, peak amplitude, crest factor, zero-crossing rate, rule-based classification.

1. Introduction

Digital Signal Processing (DSP) uses numerical operations on sampled signals to measure, transform, and interpret signal properties. Acoustic signals can be described by features such as average energy, peak magnitude, and the frequency with which a waveform crosses zero. These features can be used as inputs to simple classification rules.

The current prototype uses synthetic signals rather than a microphone or recorded WAV files. This keeps inputs reproducible and allows the processing logic to be tested independently of an audio acquisition interface.

2. Objectives

• Generate three representative synthetic signal patterns in C.

• Store samples in a fixed-size floating-point array.

• Calculate RMS amplitude, peak amplitude, crest factor, and zero-crossing rate.

• Apply threshold-based rules to assign a pattern label.

• Run the program on the TMS320C6745 DSP platform using CCS and inspect console output.

3. Hardware and Software Requirements

• DSP platform: Texas Instruments TMS320C6745 DSP kit.

• Development environment: Code Composer Studio (CCS).

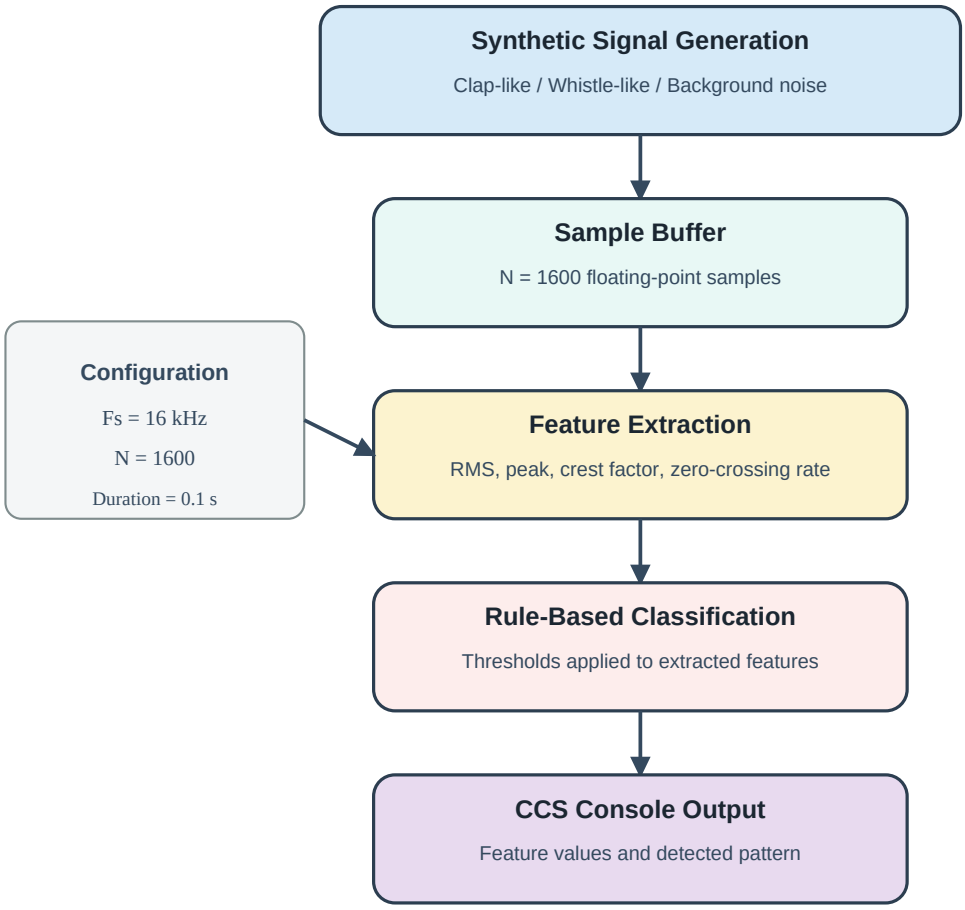
• Programming language: C, using expf(), sinf(), sqrtf(), and fabsf().

• Input: synthetic signals generated directly in the program.

• Output: feature values and classification labels printed to the CCS console.

4. System Architecture

The architecture follows a sequential processing pipeline. The program generates one signal at a time and stores it in the samples array. The feature-extraction function processes all 1,600 samples and returns four features. The classifier applies fixed thresholds, and the result is displayed in the CCS console.



Processing flow of the implemented synthetic-signal prototype

Figure 1. System architecture for the implemented synthetic-input prototype.

Configuration: sampling frequency = 16 kHz; sample count = 1,600; duration = 0.1 seconds.

5. Signal Generation

5.1 Clap-like signal: a combination of 1,700 Hz and 3,100 Hz sinusoidal components multiplied by an exponential decay. It is nonzero only during the first 0.035 seconds.

5.2 Whistle-like signal: a 2,200 Hz sine wave with amplitude 0.55 across the full 0.1-second sample buffer.

5.3 Background noise: a low-amplitude pseudo-random sequence scaled by 0.035. The fixed seed makes the sequence reproducible under consistent integer behavior.

6. Feature Extraction

RMS amplitude: RMS = sqrt((1/N) × Σ x[n]²). It summarizes the signal's effective magnitude.

Peak amplitude: Peak = max(|x[n]|). It is the largest absolute sample value.

Crest factor: Peak / RMS when RMS is nonzero. It describes the size of the maximum peak relative to RMS.

Zero-crossing rate: number of sign transitions / (N – 1). The program normalizes the crossing count by the number of adjacent-sample pairs.

7. Rule-Based Classification

The classifier first checks whether peak amplitude is greater than 0.45 and crest factor is greater than 4.0; if so, the signal is labelled CLAP-LIKE. Otherwise, if RMS is greater than 0.15, crest factor is less than 2.0, and zero-crossing rate is greater than 0.15, it is labelled WHISTLE-LIKE. All other signals are labelled BACKGROUND-LIKE.

These are heuristic thresholds for the generated signals. They are not a trained machine-learning model and have not been validated for real-world acoustic events.

8. Program Execution Flow

1. Initialize the program and print the project heading.

2. Generate the clap-like signal and calculate its features.

3. Classify the clap-like signal and print the results.

4. Generate and process the whistle-like signal.

5. Generate and process the background-noise signal.

6. Print the completion message and remain in an infinite loop.

9. Results and Output

The CCS console displays the sampling frequency, sample count, duration, four feature values, and classification label for each generated signal. The intended labels are CLAP-LIKE, WHISTLE-LIKE, and BACKGROUND-LIKE for the corresponding generated inputs.

Insert the actual measured values and screenshots from your CCS run in this section or keep them in the repository's results/ folder. This report does not fabricate numerical results.

[Insert screenshot: synthetic clap output]

[Insert screenshot: synthetic whistle output]

[Insert screenshot: synthetic background output]

10. Limitations

• The program generates synthetic samples; it does not acquire live audio from a microphone.

• The generated signals are illustrative patterns, not guaranteed models of real claps, whistles, or ambient noise.

• Classification uses fixed thresholds and has not been evaluated against a labelled real-world dataset.

• Output depends on the signal definitions and configured thresholds.

11. Future Enhancements

• Add recorded WAV input or real-time audio acquisition through a suitable interface.

• Evaluate feature calculations using real acoustic samples.

• Tune thresholds using measured data and report classification accuracy.

• Add spectral descriptors and optimize processing for embedded real-time operation.

12. Conclusion

The project demonstrates a basic DSP workflow on the TMS320C6745 platform: synthetic signal generation, time-domain feature extraction, threshold-based classification, and console reporting. It provides a foundation for further work in embedded DSP and acoustic analysis. A future version can accept recorded or live audio and evaluate the classification rules using real-world data.

Appendix A: Main Functions

pseudo_noise(): generates deterministic pseudo-random values for background noise.

generate_clap(): fills the sample array with a decaying multi-tone transient.

generate_whistle(): fills the sample array with a 2,200 Hz sinusoid.

generate_background(): fills the sample array with scaled pseudo-random noise.

extract_features(): calculates RMS, peak, crest factor, and zero-crossing rate.

classify_signal(): applies threshold rules and returns a pattern label.

run_test(): prints signal details and analysis results to the CCS console.

main(): runs the three signal tests sequentially.